

Real-Time Motor Fatigue Detection using Hardware- Accelerated Signal Processing and ML on PYNQ Z2

Leveraging HW/SW Co-Design for Edge-Based Condition Monitoring

Soham Pramanik, Jadavpur University
Prof. Sayan Chatterjee, Jadavpur University
Prof. Swagata Mondal, Calcutta University

The Problem: Why Monitor Motor Health?

Motors are Ubiquitous & Critical:

- Found in countless industrial, commercial, and consumer applications (manufacturing, robotics, HVAC, transport)

Failures are Costly:

- Unexpected downtime leads to production loss
- Expensive repairs or replacements
- Potential for secondary damage or safety hazards

"Fatigue" - The Precursor to Failure:

- Subtle degradation (wear, stress, imbalance) precedes catastrophic failure. Early detection is key

The Need for Edge Intelligence:

- Real-time, on-device analysis is required due to bandwidth, latency, and security constraints

Our Goal: Predictive Maintenance at the Edge

Objective:

- Develop a system to detect motor fatigue in real-time using electrical current signatures.

Approach:

- Acquire 3-phase current signals in real time.
- Apply advanced signal processing to extract relevant features.
- Use machine learning to classify motor state (Healthy vs. Fatigued).

Platform:

- Utilize the PYNQ Z2 board (SW on ARM cores & HW on FPGA).

Focus:

- Compare performance of SW vs. HW to showcase benefits of hardware acceleration.

Key Concepts: Understanding the Signals

3-Phase Motor Currents:

- AC current whose magnitude, shape, and frequency vary with load and motor health

Motor Current Signature Analysis (MCSA):

- Analyzing current signals to diagnose motor condition

Park's Transformation:

- Transforms 3-phase signals to a rotating reference frame (yielding I_d and I_q)
 - I_d (Direct Axis): Related to magnetic flux
 - I_q (Quadrature Axis): Related to torque

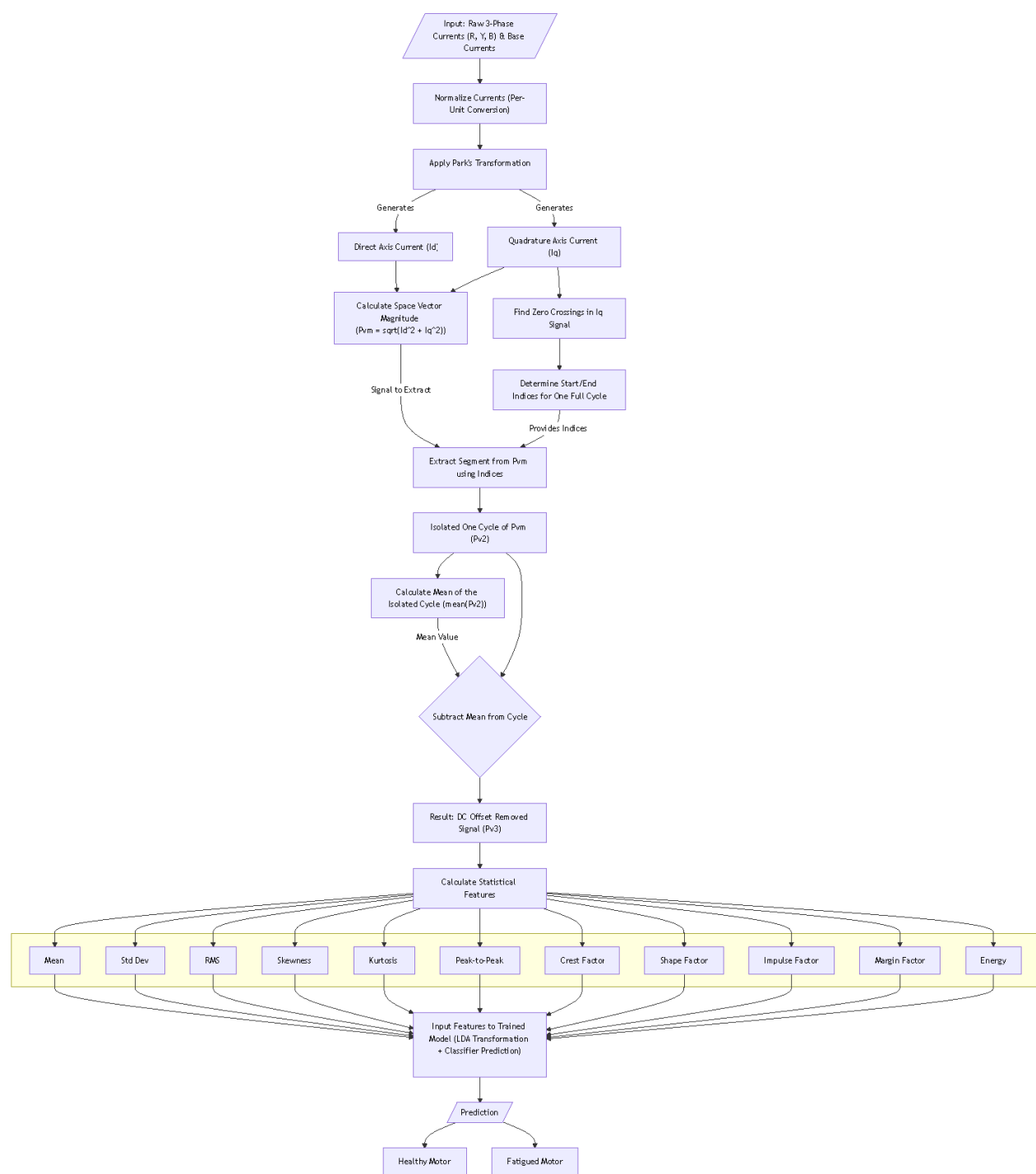
Statistical Features:

- Metrics (Mean, RMS, Skewness, Kurtosis, etc.) that capture subtle changes indicative of fatigue

The Algorithm: From Raw Current to Features

Main stages:

- Input & Normalize: Standardize current readings.
- Park's Transform: Convert to I_d and I_q
- Space Vector Mag (Pvm): Compute current magnitude (I_d and I_q)
- Cycle Isolation: Use I_q zero-crossings to isolate one representative cycle
- DC Offset Removal: Center the cycle waveform around zero
- Feature Extraction: Derive statistical features from the cycle



Implementation: HW/SW Co-Design on PYNQ Z2

Platform: Xilinx PYNQ Z2

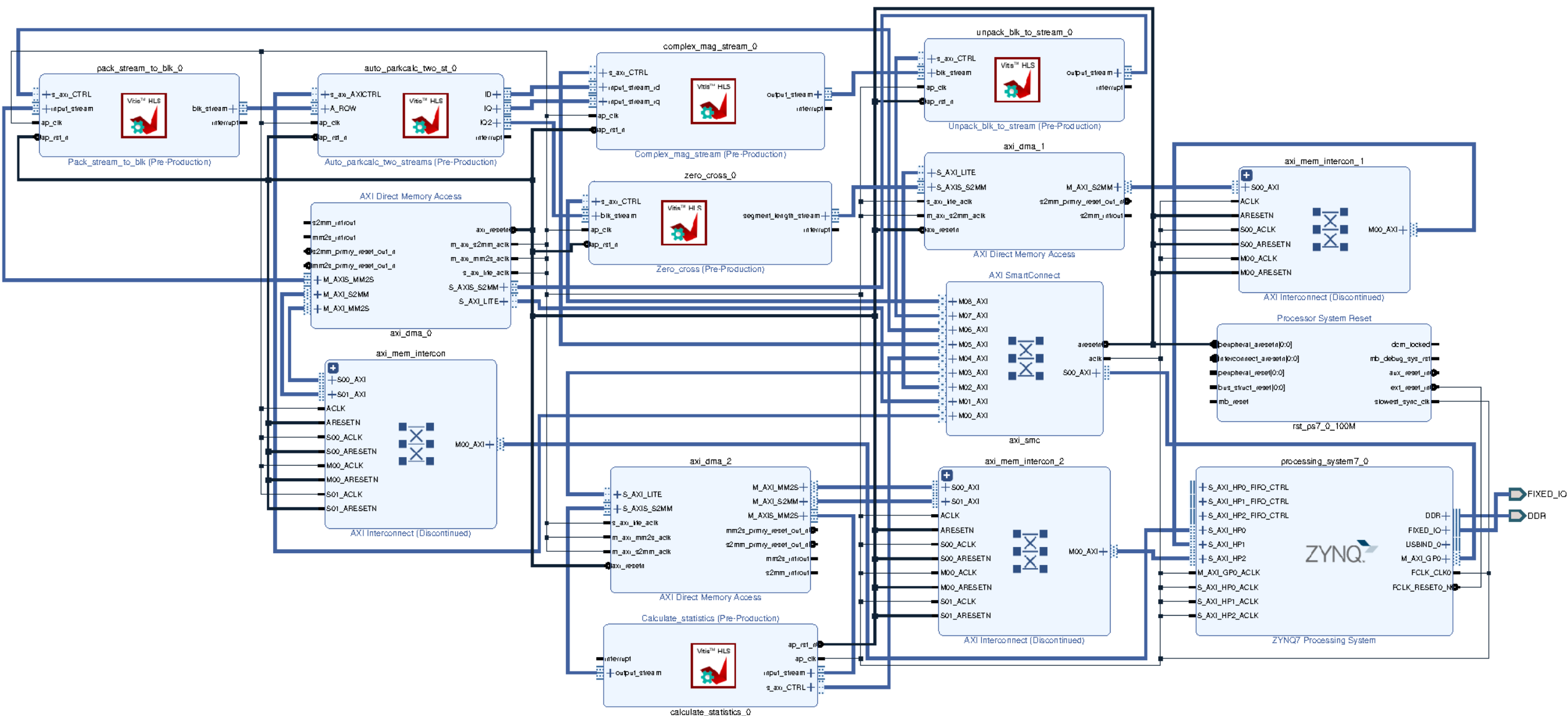
- Processing System (PS): Dual-core ARM Cortex-A9 (Linux, Python – Software Implementation)
- Programmable Logic (PL): FPGA Fabric (Hardware Implementation)

Workflow:

- The algorithm was implemented twice:
 - Software: Python on ARM cores
 - Hardware: Custom logic on FPGA (HLS)

Goal:

- Directly compare execution speed between SW and HW implementations.





```
TheFinalDance/ x MotorFatigue - jupyter No: x +
172.24.49.236:9090/notebooks/TheFinalDance/MotorFatigue.ipynb
AlmaLinux Documentation Blog Bug tracker GitHub organization
jupyter MotorFatigue Last Checkpoint: 2 minutes ago (autosaved)
Not Trusted Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
In [14]: %time
dma2_send.transfer(Stat_buffer_input)
dma2_recv.transfer(Stat_buffer_output)
dma2_send.wait()
dma2_recv.wait()
CPU times: user 681 µs, sys: 50 µs, total: 731 µs
Wall time: 775 µs

In [15]: for i in range(11): print(Stat_buffer_output[i])
-0.76991e-09
0.0128521
0.517018
0.585691
0.052237
0.0128521
2.58931
1.21486
3.14566
317.084
0.058828

In [16]: def calc_statistics(data):
# Number of data points
n = len(data)

# Calculate mean
mean_val = np.mean(data)

# Calculate standard deviation
std_dev = np.std(data)

# Find min and max
min_val = np.min(data)
max_val = np.max(data)

# Calculate RMS
rms_val = np.sqrt(np.mean(np.square(data)))

# Calculate absolute mean for shape and impulse factors
mean_abs = np.mean(np.abs(data))

# Calculate skewness and kurtosis
```



Results: Hardware Acceleration Performance

Execution time for feature extraction pipeline:

- Software (ARM A9 - Python): 20.46 ms
- Hardware (FPGA Fabric): 1.531 ms

Speedup:

~13.36x faster (20.46 / 1.531)

Gains & Benefits of Hardware Acceleration

- Significant Speedup (~13.36x)
- Lower Latency for rapid detection
- Enables true edge processing:
 - Reduces data transmission requirements
 - Enhances privacy and security
- CPU Offloading: ARM cores can run the final ML model and handle communications
- Potential for improved power efficiency

Use Cases & Applications

- **Predictive Maintenance:** Schedule maintenance based on detected fatigue
- **Industrial Automation:** Monitor motors in assembly lines, CNC machines, robotic arms
- **HVAC Systems:** Monitor fans, pumps, and compressors in large buildings
- **Renewable Energy:** Monitor generators in wind turbines or solar arrays
- **Smart Edge Devices:** Integrate into smart sensors or embedded monitoring systems
- **Automotive** (Non-critical): Monitor auxiliary motors (pumps, fans) in vehicles.

Conclusion

- Developed an effective signal processing pipeline to extract features indicative of motor fatigue.
- Successfully implemented the pipeline on the PYNQ Z2 using both software and hardware approaches.
- Achieved a ~8.5x performance improvement via hardware acceleration.
- Demonstrated the benefits of HW/SW co-design for real-time, edge-based condition monitoring.